Dylan Thornburg
9/29/2025

CSEN 127 HW2

1. A 4-bit comparator has two 4-bit unsigned inputs, A and B. It generates three outputs. We'll call them L, E, and G. L asserts when A<B, E asserts when A==B, and G asserts when A>B. It also has three additional inputs (we'll call them LIN, EIN, and GIN) that allow for cascading these 4-bit comparators by connecting LIN, EIN, and GIN on the high 4 bits to L, E, and G for the low 4 bits.

(a) Write a Verilog module for one of these 4-bit comparators. You can use the comparison operators (< and >) in your code; you don't have to work out the bitwise logic. Make sure the functionality incorporates the LIN, EIN, and GIN inputs. Read part (c) to make sure you fully understand the Context.

```
module four_bit_comp(A,B,L,E,G,Lin,Ein,Gin);
        input [3:0] A;
        input [3:0] B;
        output L, E, G;
        input Lin, Ein, Gin;
        assign L = A<B || (A==B && Lin);
        assign G = A>B || (A==B && Gin);
        assign E = (A==B && Ein);
endmodule
```

(b) The outputs can be undefined for inputs that would not make sense.
W

(c) Create an 8-bit comparator module that instantiates two of your 4-bit modules. When you instantiate the low half, the LIN, EIN, and GIN inputs can be tied high or low, as you like, as long as it operates in a sensible Manner.

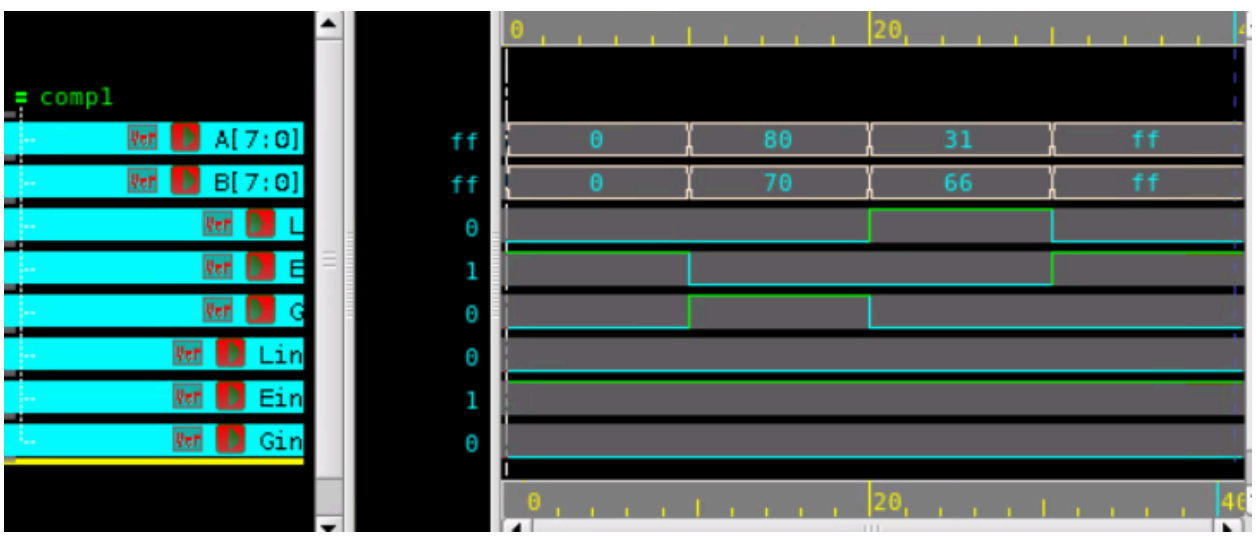
```
module eight_bit_comp(A,B,L,E,G,Lin,Ein,Gin);
        input [7:0] A;
        input [7:0] B;
        output L, E, G;
        input Lin, Ein, Gin;
        wire Lw, Ew, Gw;
        four_bit_comp low(A[3:0], B[3:0], Lw, Ew, Gw, Lin, Ein, Gin);
        four_bit_comp high(A[7:4], B[7:4], L, E, G, Lw, Ew, Gw);
endmodule
```

(d) I recommend that you simulate and test your design before turning it in.

Base Lin, Ein, Gin for testbench = Lin=0, Ein=1, Gin=0

TESTBENCH CODE:

```
module testbench();
        reg[7:0] a,b;
        reg lin, ein, gin;
        wire l, e, g;
        eight_bit_comp comp1(a, b, l, e, g, lin, ein, gin);
        initial begin
                $display("Start of the 8-bit comp test");
                $monitor("a=%d,b=%d,l=%d,e=%d,g=%d,lin=%d,ein=%d,gin=%d @
%d", a,b,l,e,g,lin,ein,gin,$time);
                a=8'b0; b= 8'b0; lin = 1'b0; ein = 1'b1; gin = 1'b0;
                #10;
                a=8'b10000000; b= 8'b01110000; lin = 1'b0; ein = 1'b1; gin = 1'b0;
                #10;
                a=8'b00110001; b= 8'b01100110; lin = 1'b0; ein = 1'b1; gin = 1'b0;
                #10;
                a=8'b11111111; b= 8'b11111111; lin = 1'b0; ein = 1'b1; gin = 1'b0;
                #10;
                $display("End of the test for 8-bit comp");
                $finish;
        end
endmodule
```



2. We are interested in a clocked 8-bit shift register with an active-low, asynchronous clear (CLRN) and parallel load capability. In addition to the CLK, CLRN and 8-bit D input, there is a 2-bit control bus S[1:0], and serial inputs for the two directions of shifting: LSI and RSI. If S[1:0] is 2'b11, the parallel load capability is used, and if S[1:0] is 2'b00 then the value holds. Otherwise, the value is shifted either left or right (I'll let you choose which bit of S controls which direction).

(a) Write a Verilog module for this 8-bit register.

```
module eight_bit_shift_reg (
	input [7:0] D,
	input clk,
	input clrn,
	input RI,
	input LI,
	input [1:0] S,
	output reg [7:0] out
);
	always @(negedge clk) begin
		if(!clrn) begin
			out=8'b00000000;
		end else begin
			case(S)
			2'b01 : begin //left shift
			out = {out[6:0],LI};
			end

			2'b10 : begin //right shift
			out = {RI,out[7:1]};
			end

			2'b11 : begin //parallel load
			out = D;
			end

			default: begin
			out=out;
			end
			endcase
		end
	end
endmodule
```

```
1 module and 0 UDP read.
recompiling module testbench
rm -f _cuarc*.so _csrc*.so pre_vcsobj_*.so share_vcsobj_*.so
if [ -x ../simv ]; then chmod a-x ../simv; fi
g++  -o ../simv      -rdynamic  -Wl,-rpath='$ORIGIN'/simv.daidir -Wl,-rpath=./simv.daidir -Wl,-rpath=/usr/synopsys/vc
/linux64/lib  -Wl,-rpath-link=./  /usr/lib64/libnuma.so.1   objs/amcQw_d.o    _2876042_archive_1.so   SIM_l.o      rma
0_0.o            -lvirsim -lerrorinf -lsnpsmalloc -lvfs    -lvcsnew -lsimprofile -luclinative /usr/synopsys/vcs/V-202
   -Wl,-no-whole-archive         _vcs_pli_stub_.o   /usr/synopsys/vcs/V-2023.12-SP1-1/linux64/lib/vcs_save_restore_new
a -ldl  -lc -lm -lpthread -ldl
../simv up to date
CPU time: .316 seconds to compile + .164 seconds to elab + .363 seconds to link
Verdi KDB elaboration done and the database successfully generated: 0 error(s), 0 warning(s)
[dthornburg@linux402604 P2]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full64;  Sep 30 20:02 2025
Start of the 8-bit Shift-Register test
d=255,clk=0,clrn=1,ri=0,li=0,s=3,out=255 @                    0
d=255,clk=1,clrn=1,ri=0,li=0,s=0,out=255 @                   10
d=255,clk=0,clrn=1,ri=0,li=0,s=0,out=255 @                   20
d=255,clk=1,clrn=1,ri=0,li=0,s=1,out=255 @                   30
d=255,clk=0,clrn=1,ri=0,li=0,s=1,out=254 @                   40
d=255,clk=1,clrn=1,ri=0,li=0,s=2,out=254 @                   50
d=255,clk=0,clrn=1,ri=1,li=0,s=2,out=255 @                   60
d=255,clk=1,clrn=0,ri=0,li=0,s=3,out=255 @                   70
d=255,clk=0,clrn=0,ri=0,li=0,s=3,out=  0 @                   80
End of the test
$finish called from file "tb.v", line 29.
$finish at simulation time                 90
           V C S   S i m u l a t i o n   R e p o r t
Time: 90
CPU Time:      0.200 seconds;       Data structure size:   0.0Mb
Tue Sep 30 20:02:02 2025
[dthornburg@linux402604 P2]$
```

TESTBENCH CODE:

```
module testbench();
        reg[7:0] d;
        reg clk,clrn,ri,li;
        reg[1:0] s;
        wire[7:0] out;
        eight_bit_shift_reg one(d, clk, clrn, ri, li, s, out);
        initial begin
                $display("Start of the 8-bit Shift-Register test");
                $monitor("d=%d,clk=%d,clrn=%d,ri=%d,li=%d,s=%d,out=%d @ %d",
d,clk,clrn,ri,li,s,out,$time);
                d=8'b11111111; clk= 1'b0; clrn = 1'b1; ri = 1'b0; li = 1'b0; s=2'b11; //parallel
load 255
                #10;
                d=8'b11111111; clk= 1'b1; clrn = 1'b1; ri = 1'b0; li = 1'b0; s=2'b00;
                #10;
                d=8'b11111111; clk= 1'b0; clrn = 1'b1; ri = 1'b0; li = 1'b0; s=2'b00; //hold 255
                #10;
                d=8'b11111111; clk= 1'b1; clrn = 1'b1; ri = 1'b0; li = 1'b0; s=2'b01;
                #10;
                d=8'b11111111; clk= 1'b0; clrn = 1'b1; ri = 1'b0; li = 1'b0; s=2'b01; //shift left
to 254
                #10;
                d=8'b11111111; clk= 1'b1; clrn = 1'b1; ri = 1'b0; li = 1'b0; s=2'b10;
                #10;
```

```
		d=8'b11111111; clk= 1'b0; clrn = 1'b1; ri = 1'b1; li = 1'b0; s=2'b10; //shift
right back to 255
		#10;
		d=8'b11111111; clk= 1'b1; clrn = 1'b0; ri = 1'b0; li = 1'b0; s=2'b11;
		#10;
		d=8'b11111111; clk= 1'b0; clrn = 1'b0; ri = 1'b0; li = 1'b0; s=2'b11; //reset to 0
		#10;
		$display("End of the test");
		$finish;
	end
endmodule
```

(b) Write a Verilog module for a 16-bit version of this register, instantiating two of your 8-bit modules.

```
module sixteen_bit_shift_reg(
	input [15:0] D,
	input clk,
	input clrn,
	input RSI,
	input LSI,
	input [1:0] S,
	output [15:0] out
);
	wire [7:0] out_one, out_two; //tried out[7:0] and out[15:8] and didnt work
	eight_bit_shift_reg one(D[7:0],clk,clrn,D[8],LSI,S,out_one);
	eight_bit_shift_reg two(D[15:8],clk,clrn,RSI,D[7],S,out_two);
	assign out = {out_two, out_one};
endmodule
```

```
CPU time: .310 seconds to compile + .169 seconds to elab + .373 seconds to link
Verdi KDB elaboration done and the database successfully generated: 0 error(s), 0 warning(s)
[dthornburg@linux402604 P2]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full64;  Sep 30 21:31 2025
Start of the 16-bit Shift-Register test
d=65535,clk=0,clrn=1,ri=0,li=0,s=3,out=65535 @                    0
d=65535,clk=1,clrn=1,ri=0,li=0,s=0,out=65535 @                   10
d=65535,clk=0,clrn=1,ri=0,li=0,s=0,out=65535 @                   20
d=65535,clk=1,clrn=1,ri=0,li=0,s=1,out=65535 @                   30
d=65535,clk=0,clrn=1,ri=0,li=0,s=1,out=65534 @                   40
d=65535,clk=1,clrn=1,ri=0,li=0,s=2,out=65534 @                   50
d=65535,clk=0,clrn=1,ri=1,li=0,s=2,out=65535 @                   60
d=65535,clk=1,clrn=0,ri=0,li=0,s=3,out=65535 @                   70
d=65535,clk=0,clrn=0,ri=0,li=0,s=3,out=    0 @                   80
End of the test
$finish called from file "tb_two.v", line 29.
$finish at simulation time                 90
           V C S   S i m u l a t i o n   R e p o r t
Time: 90
CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
Tue Sep 30 21:31:30 2025
[dthornburg@linux402604 P2]$
```

TESTBENCH CODE:

```
module testbench();
        reg[15:0] d;
        reg clk,clrn,ri,li;
        reg[1:0] s;
        wire[15:0] out;
        sixteen_bit_shift_reg one(d, clk, clrn, ri, li, s, out);
        initial begin
                $display("Start of the 16-bit Shift-Register test");
                $monitor("d=%d,clk=%d,clrn=%d,ri=%d,li=%d,s=%d,out=%d @ %d",
d,clk,clrn,ri,li,s,out,$time);
                d=16'b1111111111111111; clk= 1'b0; clrn = 1'b1; ri = 1'b0; li = 1'b0; s=2'b11;
//parallel load 65535
                #10;
                d=16'b1111111111111111; clk= 1'b1; clrn = 1'b1; ri = 1'b0; li = 1'b0; s=2'b00;
                #10;
                d=16'b1111111111111111; clk= 1'b0; clrn = 1'b1; ri = 1'b0; li = 1'b0; s=2'b00;
//hold 65535
                #10;
                d=16'b1111111111111111; clk= 1'b1; clrn = 1'b1; ri = 1'b0; li = 1'b0; s=2'b01;
                #10;
                d=16'b1111111111111111; clk= 1'b0; clrn = 1'b1; ri = 1'b0; li = 1'b0; s=2'b01;
//shift left to 65534
                #10;
                d=16'b1111111111111111; clk= 1'b1; clrn = 1'b1; ri = 1'b0; li = 1'b0; s=2'b10;
                #10;
                d=16'b1111111111111111; clk= 1'b0; clrn = 1'b1; ri = 1'b1; li = 1'b0; s=2'b10;
//shift right back to 65535
                #10;
                d=16'b1111111111111111; clk= 1'b1; clrn = 1'b0; ri = 1'b0; li = 1'b0; s=2'b11;
                #10;
```

```
                d=16'b1111111111111111; clk= 1'b0; clrn = 1'b0; ri = 1'b0; li = 1'b0; s=2'b11;
//reset to 0
                #10;
                $display("End of the test");
                $finish;
        end
endmodule
```

(c) I recommend that you simulate and test your design before turning it in
👍